



Formal Verification Report



Aave V4 Hub

March 2026

Prepared for Aave Labs

Table of contents

Project Summary	3
Project Scope	3
Project Overview	3
Assessment Methodology	4
Formal Verification	4
Security Context	4
Collaboration and Fix Validation	5
Findings Summary	6
Severity Matrix	6
Detailed Findings	7
Medium Severity Issues	8
M-01 Share rate decreases due to rounding error in fee calculation	8
M-02 Share rate decreases to less than one due to rounding error in totalAssets	9
M-03 Spoke accounting can be broken if addSpoke() is called on an existing spoke	10
Low Severity Issues	11
L-01 Loss of assets due to a missing update to drawn index	11
L-02 Arbitrary transferFrom parameters can break protocol solvency	12
L-03 View functions don't take into account the accrued fees	13
Formal Verification	14
Verification Notations	14
General Assumptions and Simplifications	14
Properties Overview	15
Detailed Properties	16
Hub	16
P-01. Accounting & Integrity Functional level	16
P-02. Only non-zero changes	17
P-03. Validity & DoS Prevention	18
P-04. Integrity of fields	18
P-05. Integrity of aggregated fields	19
P-06. Solvency and Share Rate	20
P-07. Splitting operations	20
P-08. Accrue Integrity	21
P-09. View Functions Integrity	21
P-10. Accrue Supply Rate, Preview Functions integrity over time	22
Disclaimer	24
About Certora	24

Project Summary

Project Scope

Project Name	Repository (link)	Platform
Aave V4	https://github.com/aave/aave-v4	EVM

Project Overview

This document describes the specification and verification of **Aave V4 Hub** using the Certora Prover. The document is part of three documents describing the work undertaken from **July 28, 2025** to **March 09, 2026**.

This is one of three formal verification reports, focusing on the **Hub** component. The primary verification target is `Hub.sol`. The following internal libraries are exercised as part of the Hub verification:

```
src/hub/Hub.sol
src/hub/libraries/AssetLogic.sol
src/hub/libraries/SharesMath.sol
src/hub/libraries/Premium.sol
src/hub/HubConfigurator.sol
src/hub/AssetInterestRateStrategy.sol
```

Standalone property verification of the library contracts is documented in the companion Libraries Formal Verification Report. The proven Spoke properties are reported in the companion Spoke Formal Verification Report.

The Certora Prover demonstrated that the implementation of the Solidity contracts of the Hub is correct with respect to the formal rules written by the Certora team. During the verification process, the Certora team discovered bugs in the Solidity contracts code, as listed on the following pages.

Assessment Methodology

Formal Verification

We performed verification of the **AAVE V4** protocol using the Certora verification tool which is based on Satisfiability Modulo Theories (SMT). In short, the Certora verification tool works by compiling formal specifications written in the [Certora Verification Language \(CVL\)](#) and **AAVE's** implementation source code written in Solidity.

More information about Certora's tooling can be found in the [Certora Technology Whitepaper](#).

If a property is verified with this methodology it means the specification in CVL holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

Rules. A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.

Inductive Invariants. Inductive invariants are proved by induction on the structure of a smart contract. We use constructors as a base case, and consider all other (relevant) externally callable functions that can change the storage as step cases.

Specifically, to prove the base case, we show that a property holds in any resulting state after a symbolic call to the respective constructor. For proving step cases, we generally assume a state where the invariant holds (induction hypothesis), symbolically execute the functionality under investigation, and prove that after this computation any resulting state satisfies the invariant.

Security Context

The properties verified in this report target the subset of identified threats that can be addressed at the Hub contract level:

- accounting integrity of all Hub operations (`add`, `remove`, `draw`, `restore`, `reportDeficit`, `eliminateDeficit`, `sweep`, `reclaim`), ensuring correct state transitions and non-zero change guarantees ([P-01](#), [P-02](#));
- validity and denial-of-service prevention, ensuring only a registered spoke can act on its own position, that one spoke's operations cannot adversely affect another spoke's state, and that operation ordering across spokes does not introduce spurious reverts ([P-03](#));
- integrity of individual asset and spoke fields, including valid bounds on indices, fees, and offsets ([P-04](#)), and integrity of aggregated fields, ensuring per-spoke values sum correctly to asset-level totals ([P-05](#));
- solvency and share rate monotonicity, ensuring the Hub's external balance always covers internal accounting and the supply exchange rate never decreases ([P-06](#));
- anti-splitting guarantees, ensuring users cannot profit by splitting operations into multiple calls ([P-07](#));
- interest accrual correctness, including idempotency and index monotonicity ([P-08](#));
- view function consistency with the accrued state ([P-09](#)); and
- temporal monotonicity of share rate and preview functions ([P-10](#)).

System-level concerns such as oracle feed quality, economic attack surfaces, blockchain infrastructure risks, and admin compromise are outside the scope of this report. Spoke-level properties are verified in the companion Spoke Formal Verification Report, and library-level properties are verified in the companion Libraries Formal Verification Report.

Collaboration and Fix Validation

Collaboration with the development team runs throughout the engagement to confirm expected behaviors, clarify design assumptions, and ensure accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow-up review validates applied fixes and verifies that no regressions or secondary issues have been introduced.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	0	0	0
High	0	0	0
Medium	3	3	3
Low	3	3	3
Informational	0	0	0
Total	6	6	6

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
M-01	Share rate decreases due to rounding error in fee calculation	Medium	Fixed
M-02	Share rate decreases to less than one due to rounding error in totalAssets	Medium	Fixed
M-03	Spoke accounting can be broken if addSpoke() is called on an existing spoke	Medium	Fixed
L-01	Loss of assets due to a missing update to drawn index	Low	Fixed
L-02	Arbitrary transferFrom parameters can break protocol solvency	Low	Fixed
L-03	View functions don't take into account the accrued fees	Low	Fixed

Medium Severity Issues

M-01 Share rate decreases due to rounding error in fee calculation

Severity: Medium	Impact: High	Likelihood: Medium
Files: AssetLogic.sol	Status: Fixed	Violated Property: accrueSupplyRate

Description: Share rate, computed as `totalAssets/totalShares` is expected to be monotonic increasing. Due to a rounding error in `getFeeShares()`, it was possible that the share rate would decrease

```
JavaScript
function getFeeShares(
  ...
  uint256 feesAmount = indexDelta
    .rayMulDown(asset.baseDrawnShares + asset.premiumDrawnShares)
    .percentMulDown(liquidityFee);
```

Exploit Scenario: This computation is not consistent with the computation in `totalDebt` that computed the value of `baseDebt` and `premiumDebt` separately.

Recommendations: Rounding has to be consistent in all computations

Customer's response: Fixed the computation to be similar to one in `totalSuppliedAssets` in `v0.5.1`.

Fix Review: Fix confirmed on [PR#933](#).

M-02 Share rate decreases to less than one due to rounding error in totalAssets

Severity: Medium	Impact: High	Likelihood: Low
Files: AssetLogic.sol	Status: Fixed	Violated Property: totalAssetsVsShares_supplyExchangeRateIsMonotonic

Description: The amount of total assets is expected to be at least total shares. Due to a rounding error in `totalAddedAssets()`, it was possible that the share rate would decrease to less than one.

JavaScript

```
function totalAddedAssets(IHub.Asset storage asset) internal view returns (uint256) {
    uint256 drawnIndex = asset.getDrawnIndex();
    return
        asset.liquidity +
        asset.swept +
        asset.deficitRay.fromRayUp() +
        asset.totalOwed(drawnIndex) -
        asset.realizedFees -
        asset.getUnrealizedFees(drawnIndex);
}
```

Exploit Scenario: Function `reportDeficit()` reduces drawnShares and increases the deficit. When computing the total assets, the deficit and debt are rounded up. It could be that a report deficit moves assets from debt to deficit, and now due to rounding the total assets is reduced

Recommendations: Rounding has to be once in case of assets moving between “buckets”

Customer’s response: Fixed the computation to have just one rounding (after summing up in ray all assets)

Fix Review: Fix confirmed on [PR#1033](#).

M-03 Spoke accounting can be broken if addSpoke() is called on an existing spoke.

Severity: Medium	Impact: High	Likelihood: Low
Files: LiquidityHub.sol	Status: Fixed	Violated Property: solvency_external , sumOfSpoke*

Description: In early version of the codebase

(<https://github.com/aave/aave-v4/blob/457657bb8198523117845de8a099d0a0353dce78/src/contracts/LiquidityHub.sol#L88>) adding a spoke to the hub updates the following parameters:

JavaScript

```
_spokes[assetId][spoke] = DataTypes.SpokeData({  
    suppliedShares: 0,  
    baseDrawnShares: 0,  
    premiumDrawnShares: 0,  
    premiumOffset: 0,  
    realizedPremium: 0,  
    lastUpdateTimestamp: block.timestamp,  
    config: config  
});
```

Exploit Scenario: If the trusted entity adds a spoke that is already registered for that assetId then the new values will overwrite the existing ones and this will break the accounting and the solvency of the protocol.

Recommendations: We recommend reverting when trying to add a spoke that is already added.

Customer's response: Fixed in **v0.5.0**.

Fix Review: Fix confirmed on [PR#665](#).

Low Severity Issues

L-01 Loss of assets due to a missing update to drawn index

Severity: Low	Impact: Medium	Likelihood: Low
Files: AssetLogic	Status: Fixed	Violated Property: baseDebtIndex_increasing

Description: Accrue updates the debt index according to the change in time. When there is no debt the index does not change. The code checked only for baseDebt but not premium debt. However it is not reasonable to have only premium debt in all spokes.

JavaScript

```
function getDrawnIndex(DataTypes.Asset storage asset) internal view returns (uint256) {
    uint256 previousIndex = asset.baseDebtIndex;
    uint256 lastUpdateTimestamp = asset.lastUpdateTimestamp;
    if (lastUpdateTimestamp == block.timestamp || asset.baseDrawnShares == 0) {
        return previousIndex;
    }
}
```

Exploit Scenario: If a hub has only premium shares then no interest is accumulated.

Recommendations: Check for premium debt also.

Customer's response: Fixed.

Fix Review: Fix confirmed on [PR#665](#).

L-02 Arbitrary transferFrom parameters can break protocol solvency

Severity: Low	Impact: Low	Likelihood: Low
Files: Hub.sol	Status: Fixed	Violated Property: solvency_external

Description: Early code version of the codebase had the Hub transferring assets in and out via transferFrom instead of the Spoke.

The from and to parameters could be with arbitrary values which allowed anybody to use the existing allowances to the Hub or set both parameters to the hub address which resulted in successful `add()` and `restore()` calls thus breaking protocol solvency and accounting.

Exploit Scenario: Attacker can set the Hub as the from and to parameters and act like adding collateral but in reality he does not transfer any assets.

Recommendations: We recommend the from parameter to be the initial caller.

Customer's response: Fixed.

Fix Review: The fix properly mitigates the issue identified. The transferring of funds was moved from the Hub to the Spoke as a design decision made at a later stage.

Fix confirmed on [PR#955](#).

L-03 View functions don't take into account the accrued fees

Severity: Low	Impact: Low	Likelihood: High
Files: LiquidityHub.sol	Status: Fixed	Violated Property: viewFunctionsIntegrity

Description: In an early version of the codebase (<https://github.com/aave/aave-v4/blob/457657bb8198523117845de8a099d0a0353dce78/src/contracts/LiquidityHub.sol>) `accrue()` increases `asset.suppliedShares` by minting new shares to the `feeReceiver` based on the growth of `drawIndex`. This means that with time the total `suppliedShares` of an asset grows.

View functions such as `LiquidityHub.getAssetSuppliedShares()` and `LiquidityHub.getAssetSuppliedAmount()` however do not take this into consideration. When called they will always return outdated amounts.

Exploit Scenario: Off-chain services or contracts that use the view functions to make calculations can be confused.

Recommendations: We recommend calculating the accrued fees in the view functions in order for them to provide up to date values.

Customer's response: Fixed.

Fix Review: Fixed confirmed on [PR#540](#).

Formal Verification

Verification Notations

Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Formally Verified After Fix	The rule was violated due to an issue in the code and was successfully verified after fixing the issue

General Assumptions and Simplifications

- We use our own implementation of `Math.mulDiv`, `Math.mulDivUp` and `Math.mulDivDown`. These are equivalent to the originals and better suited for verification purposes.
- Loops are inherently difficult for formal verification. We handle loops by unrolling them a specific amount of times. We use a **loop_iter of 3**, unrolling each loop up to three times.
- Underlying assets are assumed to be standard ERC20, specifically:
 - a. No fees on transfer
 - b. Without double entries
 - c. Decimals between 6 and 18
- The price of assets is assumed to be non-zero.
- The verified Solidity contracts are compiled with solc8.28 (with via-ir).

Properties Overview

ID	Contract	Title	Impact
P-01	Hub	Functional integrity	Direct fund loss, broken economic model, potential insolvency
P-02	Hub	Only non-zero changes	Direct fund loss, broken model, potential insolvency
P-03	Hub	Validity & DoS Prevention	Integrity of data and denial of service
P-04	Hub	Integrity of fields	fund loss, broken economic model, potential insolvency
P-05	Hub	Integrity of aggregated fields	fund loss, broken economic model, potential insolvency
P-06	Hub	Solvency and Share Rate	Invalid changes
P-07	Hub	Splitting operations	fund loss, potential insolvency
P-08	Hub	Accrue Integrity	Interest loss
P-09	Hub	View functions integrity	Integrity of data and denial of service
P-10	Hub	Accrue Supply Rate	Interest loss

Detailed Properties

Hub

Safe Assumptions and modularity:

We prove properties group P01- P08 with assumption that `accrue()` has been called on this block. This is a safe assumption as we prove that `accrue()` is always called before any change to position, and we prove in P-09 and P-10 that accrue preserves the state as expected.

P-01. Accounting & Integrity Functional level

Status: Formally Verified

Rule Name	Status	Description	Link to rule report
add_integrity	Formally Verified	Add operation increases liquidity and decreases spoke added shares	Run link
remove_integrity	Formally Verified	Remove operation decreases drawn shares, premium shares, premium offset, deficit ray, spoke added shares, liquidity, external balance, and to balance	
draw_integrity	Formally Verified	Draw operation decreases drawn shares, premium ray, spoke added shares, deficit ray, liquidity, external balance, and to balance	
restore_integrity	Formally Verified	Restore operation decreases debt shares, premium ray, deficit ray, spoke added shares, liquidity	
reportDeficit_integrity	Formally Verified	Report deficit operation decreases drawn shares, premium ray, deficit ray, spoke added shares, liquidity	
eliminateDeficit_integrity	Formally Verified	Eliminate deficit operation increases spoke added shares, liquidity, drawn shares, premium	

		<i>shares, premium offset, and deficit ray</i>	
sweep_integrity	Formally Verified	<i>Sweep operation increases liquidity, external balance, to balance, swept, spoke added shares, drawn shares, premium shares, premium offset, and deficit ray</i>	
reclaim_integrity	Formally Verified	<i>Reclaim operation decreases liquidity, swept, spoke added shares, drawn shares, premium shares, premium offset, and deficit ray</i>	

P-02. Only non-zero changes

Status: Formally Verified

Rule Name	Status	Description	Link to rule report
nothingForZero_add	Formally Verified	<i>Add operation increases external balances and increases internal accounting while decreasing from balance</i>	Run link
nothingForZero_remove	Formally Verified	<i>Remove operation decreases external balances and decreases internal accounting while increasing to balance</i>	
nothingForZero_draw	Formally Verified	<i>Draw operation increases debt shares and transfers assets to recipient</i>	
nothingForZero_eliminateDeficit	Formally Verified	<i>Report deficit operation decreases debt shares and increases liquidity</i>	
nothing_for_zero_sweep	Formally Verified	<i>Sweep operation increases liquidity and decreases swept</i>	
nothing_for_zero_reclaim	Formally Verified	<i>Reclaim operation increases liquidity and decreases swept</i>	

P-03. Validity & DoS Prevention

Status: Formally Verified After Fix

Rule Name	Status	Description	Link to rule report
reportDeficitSameAsPreviewRestoreByAssets	Formally Verified	ReportDeficit return same value as previewRestoreByAssets	Run link
validSpokeOnly	Formally Verified	Only a valid spoke can perform changes to the spoke position (except for receiving fee shares)	
frontRunOnRefreshPremium	Formally Verified After Fix	Order of refreshPremium of two different spokes should not change reverting cases. This rule was priorly violated due to a known issue	
noChangeToOtherSpoke	Formally Verified	Can only increase a spoke's asset or decrease debt	Run link

P-04. Integrity of fields

Status: Formally Verified

Rule Name	Status	Description	Link to rule report
validAssetId	Formally Verified	Integrity of a validAsset	Run link
drawnIndexMin	Formally Verified	DrawnIndex is greater than or equal to RAY on listed assets	
liquidityFee_upper_bound	Formally Verified	LiquidityFee upper bound: config.liquidityFee must not exceed PercentageMathExtended.PERCENTAGE_FACTOR	

premiumOffset_Integrity	Formally Verified	PremiumOffsetRay must not exceed the premiumShares on the asset and spoke level	
assetToSpokesIntegrity	Formally Verified	_assetToSpokes integrity: indexes and values always match	

P-05. Integrity of aggregated fields

Status: Formally Verified After Fix

These properties were violated due to [M-03](#).

Rule Name	Status	Description	Link to rule report
sumOfSpokeSupplyShares	Formally Verified After Fix	The sum of hub._spokes[assetId][spoke].addedShares for all spoke equals to hub._assets[assetId].addedShares	Run link
sumOfSpokeDrawnShares	Formally Verified After Fix	The sum of hub._spokes[assetId][spoke].drawnShares for all spoke equals to hub._assets[assetId].drawnShares	
sumOfSpokePremiumDrawnShares	Formally Verified After Fix	The sum of hub._spokes[assetId][spoke].premiumShares for all spoke equals to hub._assets[assetId].premiumShares	
sumOfSpokePremiumOffset	Formally Verified After Fix	The sum of hub._spokes[assetId][spoke].premiumOffsetRay for all spoke equals to hub._assets[assetId].premiumOffsetRay	
sumOfSpokeDeficit	Formally Verified After Fix	The sum of hub._spokes[assetId][spoke].deficitRay for all spoke equals to hub._assets[assetId].deficitRay	

P-06. Solvency and Share Rate

Status: Formally Verified After Fix

These properties were violated due to [M-02](#), [M-03](#) and [L-02](#).

Rule Name	Status	Description	Link to rule report
solvency_external	Formally Verified After Fix	External balance is at least as internal accounting. This property was violated due to M-03 and L-02	Run link
totalAssetsVsShares	Formally Verified After Fix	The sum of added assets is greater than or equal to the sum of added shares. This property was violated due to M-02	Run link eliminateDeficit
supplyExchangeRatesMonotonic	Formally Verified After Fix	Supply rate is never decreasing. This property was violated due to M-02	Run Link restore() eliminateDeficit

P-07. Splitting operations

Status: Formally Verified

Rule Name	Status	Description	Link to rule report
addAdditivity	Formally Verified	Adding in two steps is less beneficial to the user than doing it in one step	Run link
drawAdditivity	Formally Verified	Drawing in two steps is less beneficial to the user than doing it in one step	Run link
restoreAdditivity	Formally Verified	Restoring in two steps is less beneficial to the user than doing it in one step	Run link

reportDeficitAdditivity	Formally Verified	Reporting deficit in two steps is less beneficial to the user than doing it in one step	Run link
eliminateDeficitAdditivity_DeficitRay	Formally Verified	Eliminating deficit in two steps is the same as doing it in one step, with respect to deficit	Run link

P-08. Accrue Integrity

Status: Formally Verified After Fix

Rule Name	Status	Description	Link to rule report
runningTwicelsEquivalentToOne	Formally Verified	Two invocations of accrue() at the same block result in a state exactly the same as the first execution	Run link
baseDebtIndexMin_accrue	Formally Verified	Once baseDebtIndex is set it is at least RAY	
lastUpdateTimestamp_updatedToCurrentBlockTimestamp	Formally Verified	lastUpdateTimestamp is updated to the current block timestamp	
noChangeToOtherFields_accrue	Formally Verified	When accrue is called, no change to other fields beside lastUpdateTimestamp, drawnIndex and realizedFees	
baseDebtIndex_increasing	Formally Verified After Fix	BaseDebtIndex is increasing on block change when baseRate is at least SECONDS_PER_YEAR and index is set. This property was violated due to L-01	
premiumOffset_Integrity_accrue	Formally Verified	Prove premiumOffsetRay is always less than or equal to premiumShares * drawnIndex	

P-09. View Functions Integrity

Status: Formally Verified After Fix

Rule Name	Status	Description	Link to rule report
viewFunctionsIntegrity	Formally Verified After Fix	View functions are isomorphic to accrue, they return the same value if accrue was called or not. This property was violated due to L-03	Run link
viewFunctionsRevertIntegrity	Formally Verified	View functions revert under the same state if accrue is called or not called before the view function is called	

P-10. Accrue Supply Rate, Preview Functions integrity over time

Status: Formally Verified After Fix

Rule Name	Status	Description	Link to rule report
accrueSupplyRate	Formally Verified After Fix	accrue cannot decrease the share rate. This property was violated due to M-01	Run link
shareRate_withoutAccrue_time_monotonic	Formally Verified	Share rate is monotonic over time without accrue	Run link
previewRemoveByShares_withoutAccrue_time_monotonic	Formally Verified	Preview remove by shares is monotonic over time without accrue	Run link
previewAddByAssets_withoutAccrue_time_monotonic	Formally Verified	Preview add by assets is monotonic over time without accrue	Run link
previewAddByShares_withoutAccrue_time_monotonic	Formally Verified	Preview add by shares is monotonic over time without accrue. Proved in two steps with some limitation on liquidityFee	Run link part 1 Run link part 2
previewRemoveByAssets_withoutAccrue_time_monotonic	Formally Verified	Preview remove by assets is monotonic over time without accrue	Run link

previewDrawByAssets_without Accrue_time_monotonic	Formally Verified	Preview draw by assets is monotonic over time without accrue	Run link
previewDrawByShares_without Accrue_time_monotonic	Formally Verified	Preview draw by shares is monotonic over time without accrue	Run link
previewRestoreByAssets_witho utAccrue_time_monotonic	Formally Verified	Preview restore by assets is monotonic over time without accrue	Run link
previewRestoreByShares_witho utAccrue_time_monotonic	Formally Verified	Preview restore by shares is monotonic over time without accrue	Run link
feeAmountIncrease	Formally Verified	Fee amount increase in accrue is equal to the unrealized fee at this timestamp	Run link
maxgetUnrealizedFees	Formally Verified	Prove that the maximum value of getUnrealizedFees is at 100% liquidityFee	Run link
lastUpdateTimestampSameAsB lockTimestamp	Formally Verified	When the lastUpdateTimestamp is the same as the block timestamp, the unrealized fees are 0	Run link

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.